VERITAS AI — VIVA & PRESENTATION GUIDE

Big Data Analytics Capstone • Complete Technical & Viva Preparation Handbook

---

1. The 2-Minute Opening Elevator Pitch

Deliver this opening statement to your external examiners at the start of your presentation:

> *"Respected examiners, Veritas AI is an Evidence-Grounded Fake News Detection platform engineered on Big Data Analytics (BDA) principles. Traditional ML classifiers operate as closed-world black boxes: they look only at text style and guess 50/50 on unseen breaking news without citing sources. In contrast, Veritas AI indexes 67,587 verified news articles and fact-checks (~3.7 GB indexes) using Apache Spark and Hadoop HDFS. When a user submits any claim, our system extracts factual entities via Llama-3.3-70B, performs dual-stage hybrid retrieval (dense multilingual vectors + sparse TF-IDF) in under 300ms, reranks candidates using a 5-signal formula, and synthesizes an evidence-grounded verdict: AUTHENTIC, FAKE, or an anti-hallucination safety state UNCERTAIN. The system is deployed with a high-performance FastAPI backend and an interactive React 19 dashboard."*

2. Big Data Analytics (BDA) Implementation Deep Dive

• **The 4 V's of Big Data**: Volume (67,587 articles, 3.7 GB matrices), Variety (Hindi/English text, social media claims, fact-check investigations), Velocity (sub-4-second retrieval across 67,587 records), and Veracity (strict evidence grounding preventing hallucination).

• **Hadoop HDFS**: Distributed block storage interface (hdfs://localhost:9000/...) storing the partitioned 365 MB multi-line news dataset.

• **Apache Spark (PySpark)**: In-memory distributed engine in spark/analyze_news.py running on local[*]. Enforces a strict StructType schema, verifies 67,587 rows, computes class distribution (45,845 Real vs 21,742 Fake), measures text length distributions, and generates 4 analytical CSV reports in results/spark/.

• **Vector Space Indexing**: Precomputed 347 MB dense semantic matrix and 3.6 GB sparse character/word TF-IDF matrices enabling instant retrieval without scanning the CSV.

3. AI & Hybrid RAG Verification Pipeline

• **Stage 1: Claim & Entity Extraction** — Llama-3.3-70B via Groq extracts core claims and named entities with exponential backoff retry.

• **Stage 2: Dual-Stage Hybrid Search** — Dense multilingual embeddings (MiniLM) and sparse TF-IDF retrieve the top 150 candidates in <300ms.

• **Stage 3: Multi-Signal Cross-Reranker** — Evaluates composite score: 0.35*Semantic + 0.20*Lexical + 0.25*Claim + 0.15*Entity + 0.05*Number to isolate the top 3-5 articles.

• **Stage 4: Grounded Reasoning** — Llama-3.3-70B outputs AUTHENTIC, FAKE, or UNCERTAIN under strict anti-hallucination constraints.

4. Step-by-Step Live Demo Script (What to Show)

Open http://127.0.0.1:8000 in your browser and demonstrate these three benchmark scenarios:

• **Demo 1 (Fake News)**: Click 'CAA Bengal Violence' -> Red badge **FAKE NEWS** (96% confidence) citing Fact-Check #57100.

• **Demo 2 (Real News)**: Click 'Misbah-ul-Haq COVID' -> Green badge **AUTHENTIC NEWS** (95% confidence) citing official news #34373.

• **Demo 3 (Uncertain)**: Click 'Modi Passing Rumor' -> Amber badge **UNCERTAIN** (90% confidence), proving the model refuses to hallucinate.

5. Top 12 Frequently Asked Viva Questions & Perfect Answers

Q1: Why did you use RAG instead of just fine-tuning a model?

Answer: Fine-tuning bakes knowledge into static weights that quickly become obsolete and still hallucinate. RAG decouples knowledge storage (our 67,587-article Big Data index) from the language model, allowing instant knowledge updates without retraining.

Q2: What is the exact role of Apache Spark?

Answer: PySpark performs distributed ETL, schema enforcement (StructType), class balance profiling (45,845 Real vs 21,742 Fake), length analytics, and exports 4 analytical CSV reports into results/spark/.

Q3: Why is your search called 'Hybrid'?

Answer: Dense vectors understand context across languages but blur exact numbers/dates. Sparse TF-IDF character n-grams match exact spelling but miss synonyms. Our dual hybrid search executes both in parallel to capture both meaning and exact keywords.

Q4: What prevents the LLM from hallucinating?

Answer: Strict evidence grounding: the LLM is provided only the top-3 reranked evidence articles from our corpus and is strictly instructed to return UNCERTAIN if proof is missing.

Q5: Why does the model output UNCERTAIN for some true real-world facts like NASA's rover landing?

Answer: Because Veritas AI is an evidence-grounded verification system, not an unconstrained chatbot. If an accredited article confirming the landing is not in our 67,587 dataset, the model correctly refuses to fabricate evidence.

Q6: How do you achieve sub-4-second latency across 67,587 articles?

Answer: Using memory-mapped vector arrays (numpy mmap_mode='r') and FastAPI's lifespan preloader that loads indexes into RAM once at startup.

Q7: What are the mathematical weights in your reranker?

Answer: Score = 0.35*Semantic + 0.20*Lexical + 0.25*Claim + 0.15*Entity + 0.05*Number.

Q8: How does the system handle Hindi and English?

Answer: We use multilingual MiniLM-L12-v2, which projects Hindi and English into the same 384-dimensional latent space, enabling cross-lingual semantic matching.

Q9: What is the frontend architecture?

Answer: Built using React 19 and Vite 8, featuring Day/Night themes in localStorage and an SVG circular gauge that dynamically calculates stroke-dashoffset.

Q10: What are the contents of results/spark/?

Answer: It contains 4 CSV reports generated by PySpark: label_distribution.csv, label_statistics.csv, length_statistics.csv, and data_quality.csv.

Q11: How would this scale to 10 million articles in production?

Answer: We would deploy a multi-node Hadoop HDFS cluster, use Apache Spark Streaming on Kafka for real-time article ingestion, and migrate dense vector storage to a distributed vector database like Milvus or Pinecone.

Q12: What is the biggest contribution of your project?

Answer: Bridging Big Data Analytics (Hadoop/Spark) with Generative AI (RAG/LLM) to deliver verifiable, anti-hallucinatory fact-checking with cited documentary proof.

6. Codebase Map: Explaining Files to Team Members

• spark/analyze_news.py: PySpark distributed analytics script computing class balances and length statistics.

• spark/preprocess.py: ETL cleaning script normalizing raw news Excel data into news_cleaned.csv.

• ai/hybrid_retriever.py: Dual-stage search combining MiniLM dense embeddings and TF-IDF matrices over 67,587 articles.

• ai/reranker.py: 5-signal cross-reranker that computes composite evidence scores.

• ai/llm_analyzer.py: Grounded reasoning engine synthesizing REAL, FAKE, or UNCERTAIN verdicts.

• backend/main.py: FastAPI server with lifespan preloading and static React mount.

• frontend/src/App.jsx: Master React component managing search state, theme, and results.